

# DiffCMR: Fast Cardiac MRI Reconstruction with Diffusion Probabilistic Models

IG3D Course — Sorbonne University — 2026

**Maël Lecene**

Student ID: 21500562

April 04, 2026

# Contents

|          |                                                                   |           |
|----------|-------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                               | <b>2</b>  |
| <b>2</b> | <b>Methods</b>                                                    | <b>2</b>  |
| 2.1      | Dataset and K-Space Processing . . . . .                          | 2         |
| 2.2      | DiffCMR Reproduction and Codebase Refactoring . . . . .           | 2         |
| 2.3      | Novel Extension via Flow Matching . . . . .                       | 3         |
| <b>3</b> | <b>Results</b>                                                    | <b>4</b>  |
| <b>4</b> | <b>Limitations</b>                                                | <b>5</b>  |
| <b>5</b> | <b>Conclusions and Future Work</b>                                | <b>5</b>  |
| <b>A</b> | <b>Data Preparation and Processing Pipeline</b>                   | <b>6</b>  |
| A.1      | K-Space Broadcasting and Under-sampling Strategy . . . . .        | 6         |
| A.2      | Independent Slice Processing and Spatial Transformation . . . . . | 6         |
| A.3      | Global Normalization . . . . .                                    | 6         |
| <b>B</b> | <b>Mathematical Foundation of Conditional Diffusion</b>           | <b>7</b>  |
| <b>C</b> | <b>Detailed Network Architecture</b>                              | <b>8</b>  |
| <b>D</b> | <b>Bonus Exploration: Flow Matching vs. Diffusion</b>             | <b>10</b> |
| D.1      | Conceptual Intuition at the Pixel Level . . . . .                 | 10        |
| D.2      | Architectural Implementation and Code Modularity . . . . .        | 10        |
| <b>E</b> | <b>Mathematical Foundation of Conditional Flow matching</b>       | <b>10</b> |
| <b>F</b> | <b>Training Dynamics and Validation</b>                           | <b>11</b> |
| F.1      | Optimization Stability for Diffusion . . . . .                    | 11        |
| F.2      | Optimization Stability for Flow Matching . . . . .                | 13        |
| F.3      | Intermediate Validation and Perceptual Evolution . . . . .        | 14        |
| <b>G</b> | <b>Results</b>                                                    | <b>15</b> |
| G.1      | Experimental Setup . . . . .                                      | 15        |
| G.1.1    | Training Configuration and Dataset . . . . .                      | 15        |
| G.1.2    | Evaluation Configuration and Metrics . . . . .                    | 16        |
| G.2      | Quantitative Results . . . . .                                    | 17        |
| <b>H</b> | <b>Ablation Study</b>                                             | <b>17</b> |
| H.1      | Impact of Inference Steps and Flow Matching Efficiency . . . . .  | 17        |
| H.2      | Impact of Ensembling Rounds . . . . .                             | 18        |
|          | <b>References</b>                                                 | <b>20</b> |

# 1 Introduction

Cardiac Magnetic Resonance (CMR) imaging is widely recognized as the gold standard for non-invasive cardiovascular assessment. However, its inherently slow data acquisition process in the k-space requires patients to endure multiple breath-holds, leading to discomfort and severe susceptibility to motion artifacts. To accelerate this process, k-space under-sampling is standard clinical practice. Unfortunately, this acceleration violates the Nyquist-Shannon sampling theorem, introducing complex aliasing artifacts and structural noise in the reconstructed image domain.

While deep learning architectures like Convolutional Neural Networks (CNNs) and Generative Adversarial Networks (GANs) have been deployed to reconstruct fully-sampled images, a critical gap remains. Traditional deterministic models frequently suffer from regression-induced over-smoothing, while adversarial models are prone to mode collapse and the hallucination of non-existent high-frequency details, which severely compromises diagnostic reliability.

Addressing this fundamental limitation, this project explores the **DiffCMR** framework originally proposed by Xiang et al. (MICCAI 2023) [1]. Our primary objective is to harness Conditional Denoising Diffusion Probabilistic Models (DDPMs) [2] to iteratively recover high-fidelity anatomical structures from pure noise, conditioned on the aliased inputs. Furthermore, we identify the prohibitive inference time of traditional DDPMs as a critical bottleneck for clinical viability. To solve this, we propose a novel extension using **Flow Matching**—an optimal transport paradigm designed to straighten the generative trajectory—aiming to drastically accelerate the reconstruction process without sacrificing structural accuracy.

## 2 Methods

### 2.1 Dataset and K-Space Processing

The project utilized the complex 5D multi-coil cine cardiac MRI dataset from the CM-RxRecon challenge. To simulate an accelerated clinical acquisition, we applied a retrospective 2D Cartesian under-sampling mask (Acceleration Factor 4x). The raw data was then transitioned from the frequency domain to the spatial domain via an Inverse Fast Fourier Transform (`ifft2`) and combined using the Root Sum of Squares (RSS) algorithm. To ensure numerical stability during neural network training, the resulting magnitude images were globally normalized using their 99.9th percentile intensity value and strictly clipped to a  $[0.0, 1.0]$  range. This pipeline generates the highly aliased spatial domain images used as conditioning inputs for our generative models. For a comprehensive step-by-step breakdown of the k-space broadcasting, the independent slice processing, and the exact mathematical formulations of the IFFT, RSS, and normalization operations, please refer to Appendix A.

### 2.2 DiffCMR Reproduction and Codebase Refactoring

Our primary method involved reproducing the DiffCMR architecture: a Conditional DDPM where a U-Net denoiser estimates injected Gaussian noise over a reverse Markov chain, conditioned directly by the under-sampled image (see the network architecture in Figure 4, and refer to Appendix B for the detailed stochastic formulas).

While the mathematical foundation of the diffusion model was preserved, handling complex 5D tensors for thousands of slices inherently poses massive computational bottlenecks. The original open-source PyTorch codebase required profound refactoring to transition from a conceptual prototype to a scalable, high-performance pipeline.

**1. Data Pipeline and I/O Optimization:** Processing thousands of high-resolution MRI slices natively creates severe Input/Output (I/O) bottlenecks. To mitigate this, we completely restructured the core data processing. We adapted the data loading scripts to support true multi-core multiprocessing. Central segments responsible for tensor slicing, masking, and IFFT2 operations were entirely rewritten to optimize RAM and CPU usage, performing element-wise operations in-place and forcing Python’s memory garbage collection to prevent memory leaks. Furthermore, we consolidated the processed images into unified, serialized `.npy` files, overhauling the custom PyTorch `Dataset` class to efficiently stream these structures without saturating disk I/O.

**2. Multi-GPU Scalability and Training Loop Enhancements:** The original core training script was strictly refactored to achieve robust Multi-GPU compatibility. We systematically audited the codebase to eliminate hardcoded device assignments (e.g., explicit `device="cuda:x"` calls), ensuring dynamic scaling across multiple accelerators (such as our 8x V100 setup) via PyTorch’s distributed frameworks. The validation loop was also fundamentally upgraded to compute standardized quantitative metrics on-the-fly and periodically save intermediate reconstructed images, allowing for real-time visual monitoring of convergence.

**3. Inference Centralization and Hyperparameter Consistency:** Critical hyperparameters governing the diffusion process (such as noise schedules and timestep embeddings) were originally hardcoded across multiple isolated modules, creating a high risk of configuration inconsistencies between training and testing. We centralized all training and inference parameters into a cohesive `argparse` configuration system, ensuring absolute deterministic consistency across the entire pipeline.

## 2.3 Novel Extension via Flow Matching

As a major extension to the original publication, we implemented **Flow Matching**. Unlike standard diffusion, which reverses a chaotic, stochastic Brownian motion requiring thousands of integration steps, our Flow Matching module is based on optimal transport principles. It learns a deterministic, straight-line vector field mapping pure noise directly to the target data distribution.

**Architectural Implementation and Code Modularity:** From a software engineering perspective, integrating a fundamentally different generative framework required careful planning to avoid cluttering the original diffusion implementation. We developed a completely new standalone Python module dedicated exclusively to the Flow Matching logic (vector field matching and ODE solving). We implemented a custom Euler Ordinary Differential Equation (ODE) solver for the generation phase (see Appendix D for the mathematical definitions).

Crucially, to maintain strict modularity, we designed an independent, specialized training script tailored specifically for the Flow Matching paradigm. This deliberate design choice guarantees that the codebase remains clean and easily maintainable, ensuring a rigorous, uncontaminated comparison between the traditional diffusion trajectory and the optimized Flow Matching path without cross-contamination of hyperparameter configurations.

### 3 Results

The models were trained on approximately 30,000 images using a high-performance cluster (8x Nvidia V100) for 20 hours (see Appendix G.1 for the comprehensive hardware and hyperparameter configurations). As illustrated in Figure 1, our reproduced DiffCMR model successfully mitigated severe aliasing artifacts, recovering high-frequency anatomical topologies such as ventricular borders with exceptional visual fidelity.

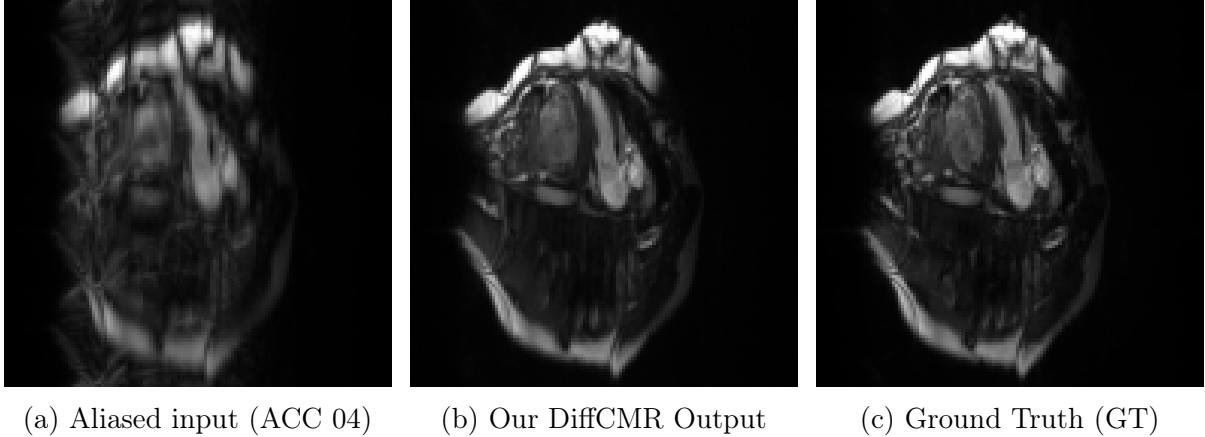


Figure 1: Representative comparison of the reconstruction pipeline. Our model successfully recovers the ground truth anatomy from the highly corrupted ACC 04 input.

Quantitatively, our reproduced Diffusion model surpassed the original paper, achieving a PSNR of 37.13 dB (vs 36.10 dB) and an SSIM of 0.9594 (vs 0.9277). However, standard diffusion inference was severely bottlenecked, taking  $\sim 105$  seconds per image for 1000 steps. By applying our **Flow Matching** approach (Table 1), we achieved comparable state-of-the-art metrics (PSNR 36.77 dB) in only **40 integration steps**, slashing the inference time to just **4.33 seconds**.

For an exhaustive breakdown of our quantitative results—including extensive comparisons with prior CNN and GAN baselines, as well as comprehensive ablation studies detailing the impact of varying integration steps ( $T$ ) and ensemble rounds ( $R$ )—please refer to Appendix G.2 and Appendix H.

Table 1: Results on Task 1-AccFactor04: Flow Matching drastically reduces inference time while preserving high quantitative fidelity.

| #Steps $T$                                  | Batch Size | Time/img | PSNR $\uparrow$ | SSIM $\uparrow$ | NMSE $\downarrow$ |
|---------------------------------------------|------------|----------|-----------------|-----------------|-------------------|
| Raw                                         | 8          | -        | 28.79           | 0.8150          | 0.2187            |
| <b>Diffusion (Their DiffCMR Original)</b>   |            |          |                 |                 |                   |
| 1000                                        | 8          | -        | 36.10           | 0.9277          | 0.0346            |
| <b>Diffusion (Our DiffCMR Reproduction)</b> |            |          |                 |                 |                   |
| 1000                                        | 8          | 105.08 s | <b>37.13</b>    | <b>0.9594</b>   | <b>0.0096</b>     |
| <b>Flow Matching (Our Novel Extension)</b>  |            |          |                 |                 |                   |
| 40                                          | 8          | 4.332 s  | 36.77           | 0.9556          | 0.0105            |

## 4 Limitations

During the reproduction phase, several challenges were encountered. The original open-source codebase contained numerous hardcoded parameters and lacked multi-GPU scalability, making the framework initially unrepeatable on modern clusters. We overcame this by entirely refactoring the training loops, separating hyperparameters into a robust `argparse` configuration, and rewriting the I/O data pipeline to optimize RAM.

The primary theoretical limitation we faced was the prohibitive inference time of the traditional DDPM (105 seconds per slice), rendering it completely unusable for real-time clinical workflows. While we overcame this by designing the Flow Matching integration, tuning the Ordinary Differential Equation (ODE) Euler solver for Flow Matching required careful balancing.

## 5 Conclusions and Future Work

This project successfully reproduced and quantitatively surpassed the DiffCMR baseline, confirming the immense potential of generative priors in resolving ill-posed MRI reconstruction tasks. Furthermore, our novel integration of Flow Matching successfully bridged the gap between high-fidelity generation and real-time clinical constraints, reducing computational time by over a factor of 200.

Due to time constraints, we were unable to fully benchmark the Flow Matching ensembling properties across higher acceleration factors (ACC08 and ACC10). Future work should focus on transitioning this 2D slice-by-slice approach into a fully spatiotemporal 3D model. Additionally, directly integrating a downstream segmentation network (like MedSAM 2) into the training loop as a perceptual loss function could further ensure that the reconstructed images are optimized not just for human vision, but for AI-driven clinical analysis.

*This appendix contains the full, detailed methodological explanations, algorithms, and comprehensive ablation studies conducted during the project.*

## A Data Preparation and Processing Pipeline

To ensure a rigorous evaluation of the DiffCMR framework, we utilized the multi-coil cine cardiac MRI dataset from the CMRxRecon challenge. The fundamental raw data resides in the frequency domain, known as the k-space. In its raw form, the fully-sampled k-space is structured as a massive, complex 5D tensor with dimensions corresponding to (frames, slices, coils,  $k_y$ ,  $k_x$ ).

### A.1 K-Space Broadcasting and Under-sampling Strategy

To simulate an accelerated clinical acquisition, we apply a retrospective 2D Cartesian under-sampling mask,  $M$ , corresponding to an acceleration factor of 4x (ACC 04). This binary mask operates exclusively on the spatial frequency dimensions ( $k_y, k_x$ ) and is computationally broadcasted across all frames, slices, and coils.

The accelerated, under-sampled k-space ( $K_{acc}$ ) is mathematically generated through a direct Hadamard (element-wise) product with the fully-sampled 5D k-space ( $K_{full}$ ):

$$K_{acc} = K_{full} \odot M$$

### A.2 Independent Slice Processing and Spatial Transformation

Due to the massive memory footprint of 5D complex tensors, the subsequent preprocessing pipeline is executed using parallel multiprocessing, operating on each 2D spatial slice independently.

Before applying the spatial transformation, both the fully-sampled and masked k-space slices are symmetrically zero-padded to a standardized target resolution of  $512 \times 512$ . Following this padding, a 2D Inverse Fast Fourier Transform (`ifft2`) is applied to the complex data of each individual coil  $c$ . This operation translates the raw frequency data  $K_c(k_x, k_y)$  into the spatial image domain  $I_c(x, y)$ :

$$I_c(x, y) = \int_{-\infty}^{+\infty} \int_{-\infty}^{+\infty} K_c(k_x, k_y) e^{i2\pi(k_x x + k_y y)} dk_x dk_y$$

To combine these multiple independent coil images into a single magnitude image—without requiring explicit sensitivity maps—we apply the Root Sum of Squares (RSS) algorithm. For a given spatial pixel  $(x, y)$  across  $N_c$  coils, the RSS reconstruction is defined as:

$$I_{RSS}(x, y) = \sqrt{\sum_{c=1}^{N_c} |I_c(x, y)|^2}$$

### A.3 Global Normalization

Finally, to ensure numerical stability during neural network training, the resulting RSS magnitude images undergo a global normalization step. Both the fully-sampled and accelerated images are divided simultaneously by the 99.9th percentile intensity value extracted

from the under-sampled image. They are subsequently clipped to a strict numerical range of  $[0.0, 1.0]$ . This operation produces the final high-fidelity ground-truth images and their corresponding highly aliased conditional inputs.

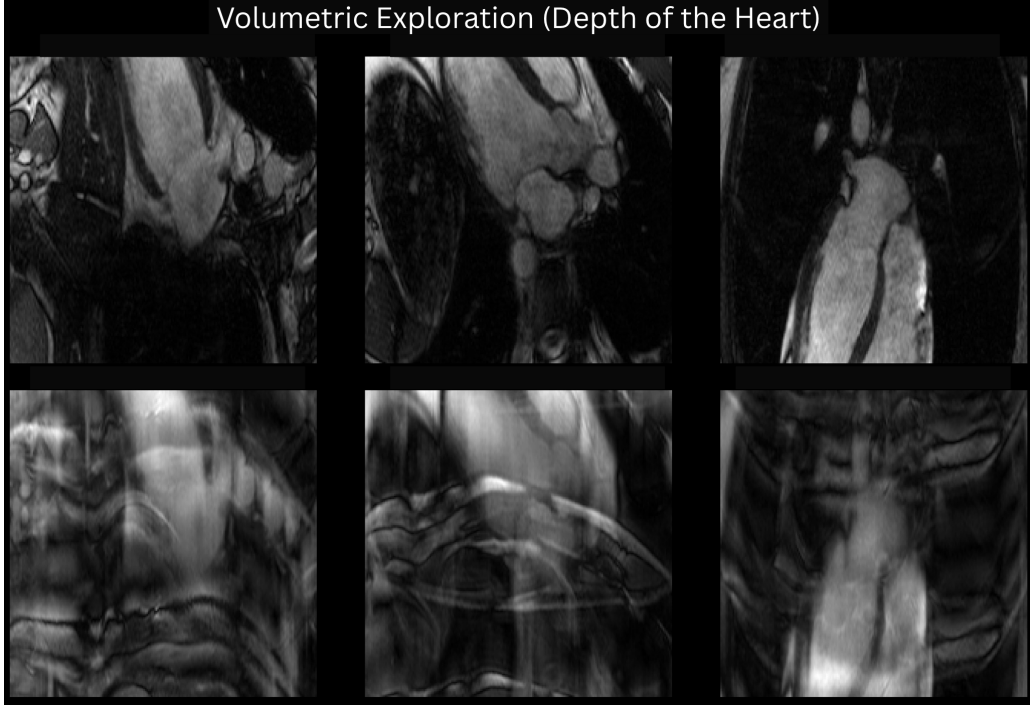


Figure 2: Visualization of the processing pipeline: The 2D ACC 04 mask applied to the fully-sampled k-space, and the subsequent conversion to the aliased image domain.

## B Mathematical Foundation of Conditional Diffusion

The core mechanism of the DiffCMR framework relies on a Conditional Denoising Diffusion Probabilistic Model (DDPM). Let  $I^f$  represent the fully-sampled ground truth image, and  $I^u$  represent the highly aliased, under-sampled image serving as the conditioning signal.

The **forward process** (or diffusion process) gradually corrupts the clean image  $I^f$  (denoted as  $x_0$ ) by adding Gaussian noise over  $T$  steps according to a variance schedule  $\beta_1, \dots, \beta_T$ . The transition probability is defined as:

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t\mathbf{I})$$

Using the reparameterization trick, we can sample  $x_t$  at any arbitrary timestep  $t$  directly in a closed form. By defining  $\alpha_t = 1 - \beta_t$  and  $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$ , the marginal probability becomes:

$$q(x_t|I^f) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}I^f, (1 - \bar{\alpha}_t)\mathbf{I})$$

Conversely, the **reverse process** aims to reconstruct the original image from pure Gaussian noise  $x_T \sim \mathcal{N}(0, \mathbf{I})$  by removing the noise step by step. In DiffCMR, this reverse Markov chain is explicitly conditioned on the under-sampled image  $I^u$ :

$$p_\theta(x_{t-1}|x_t, I^u) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t, I^u), \Sigma_\theta(x_t, t, I^u))$$



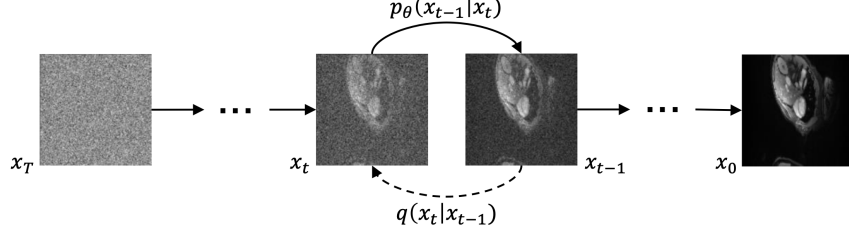


Figure 3: Forward and backward denoising diffusion processes as illustrated in the original paper.

---

**Algorithm 1** Training Algorithm

---

- 1: **Input:** total denoising steps  $T$ , under-sampled and fully-sampled image pair dataset  $D_k = \{(I_i^u, I_i^f)\}_{i=1}^{N_k}$
  - 2: **repeat**
  - 3:   Sample  $(I_i^u, I_i^f) \sim D_k$ ,  $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_{n \times n})$ ,  $t \sim \text{Uniform}(\{1, \dots, T\})$
  - 4:    $\beta_t = \frac{10^{-4}(T-t)+2 \times 10^{-2}(t-1)}{T-1}$
  - 5:    $\alpha_t = 1 - \beta_t$ ,  $\bar{\alpha}_t = \prod_{s=0}^t \alpha_s$
  - 6:    $x_t = \sqrt{\bar{\alpha}_t} I_i^f + \sqrt{1 - \bar{\alpha}_t} \epsilon$
  - 7:   Compute gradient:  $\nabla_{\theta} \|\epsilon - \epsilon_{\theta}(x_t, I_i^u, t)\|$
  - 8: **until** convergence
- 

---

**Algorithm 2** Inference Algorithm

---

- 1: **Input:** total denoising steps  $T$ , under-sampled image  $I^u$ , ensemble rounds  $R$
  - 2: **for**  $r = 1, 2, \dots, R$  **do**
  - 3:   Sample  $x_{T,r} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_{n \times n})$
  - 4:   **for**  $t = T, T-1, \dots, 1$  **do**
  - 5:     Sample  $z \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_{n \times n})$
  - 6:      $\beta_t = \frac{10^{-4}(T-t)+2 \times 10^{-2}(t-1)}{T-1}$
  - 7:      $\alpha_t = 1 - \beta_t$ ,  $\bar{\alpha}_t = \prod_{s=0}^t \alpha_s$
  - 8:      $\tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t$
  - 9:      $x_{t-1,r} = \frac{1}{\sqrt{\alpha_t}} \left( x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, I^u, t) \right) + \mathbb{K}_{[t>1]} \tilde{\beta}_t^{\frac{1}{2}} z$
  - 10:   **end for**
  - 11: **end for**
  - 12: **return**  $\sum_{r=1}^R x_{0,r} / R$
- 

## C Detailed Network Architecture

The neural network, defined by parameters  $\theta$ , is trained to estimate the injected noise  $\epsilon$ . To handle the conditioning effectively, DiffCMR defines the noise estimator network as:

$$\epsilon_{\theta}(x_t, I^u, t) = E(F(x_t) + G(I^u), H(t))$$

Where  $F$  and  $G$  are encoders for the noisy signal and the under-sampled image respectively,  $H$  encodes the timestamp, and  $E$  is the modified U-Net performing the denoising.

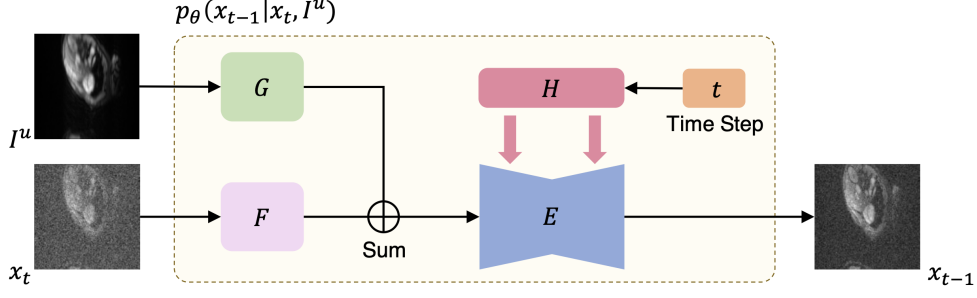


Figure 4: The pipeline of the proposed DiffCMR network, illustrating the feature summation and timestamp embedding before the U-Net ( $E$ ).

The core of this generative process heavily relies on the denoising capability of the  $E$  module. While the original DiffCMR paper does not provide a granular, layer-by-layer schematic of this specific module, based on our comprehensive analysis of the code and standard 3D diffusion paradigms, we can confidently deduce its underlying architecture.

As illustrated in Figure 5, the module operates as a robust 3D U-Net. It features a symmetric encoder-decoder structure connected by skip connections to preserve high-frequency spatial details. The network incorporates residual blocks to facilitate deep feature extraction without vanishing gradients, coupled with attention mechanisms to capture long-range dependencies across the volumetric data. Crucially, the time embedding  $H(t)$  is injected at multiple scales, allowing the network to modulate its feature maps dynamically according to the current timestep  $t$  of the diffusion or flow process.

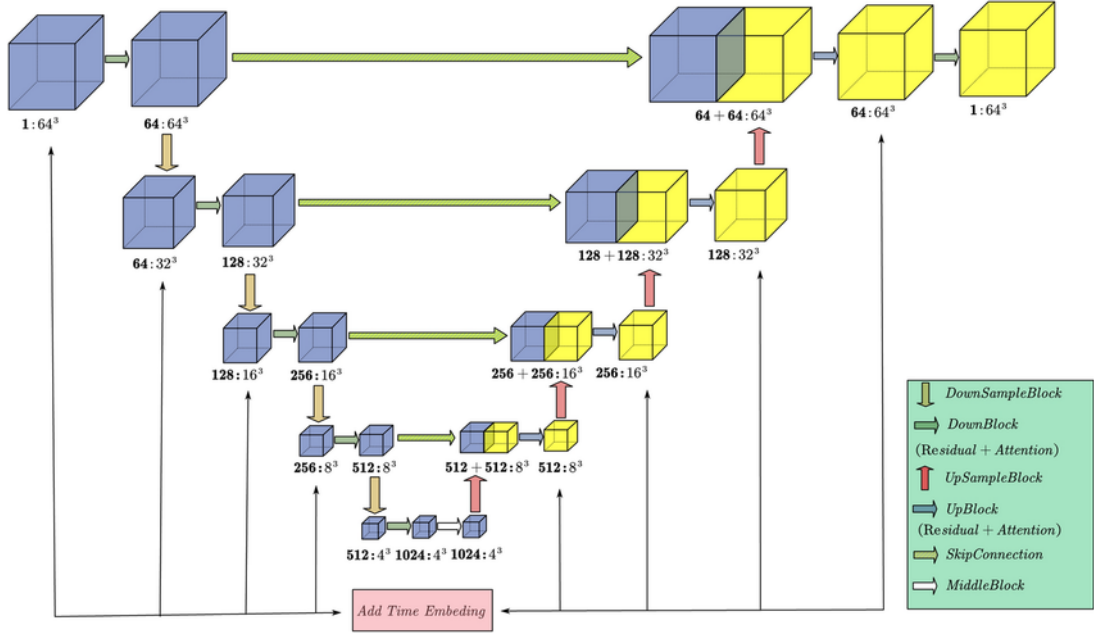


Figure 5: Detailed 3D U-Net architecture representing the  $E$  block. Reconstructed based on our analysis, it highlights the down-sampling and up-sampling paths, skip connections, and the strategic integration of residual and attention blocks alongside time embeddings.

The complete training and inference procedures, including the specific beta scheduling and the ensemble averaging during inference, are detailed in Algorithms 1 and 2.

## D Bonus Exploration: Flow Matching vs. Diffusion

Beyond the primary objective of reproducing the DiffCMR results, this project adopts a broader scientific approach by introducing a qualitative comparison between standard Diffusion Probabilistic Models and **Flow Matching**. The goal is to evaluate an alternative generative paradigm that could potentially alleviate the heavy computational burden of traditional diffusion during inference.

### D.1 Conceptual Intuition at the Pixel Level

Operating strictly at the pixel level, the core intuition behind this exploration lies in trajectory optimization. Standard diffusion models generate images by reversing a stochastic process based on Brownian motion. Because this noise-to-data mapping involves highly complex, wandering, and stochastic trajectories, the model requires an enormous number of iterative denoising steps to accurately find its way to the clean image.

In contrast, Flow Matching aims to learn a deterministic vector field based on optimal transport principles. Instead of predicting the noise at each step of a chaotic path, the Flow Matching model is trained to predict a direct, straight-line trajectory (a vector) from the pure noise distribution to the target data distribution. Because this path is fundamentally simpler and more direct, the strong hypothesis of this comparative study is that Flow Matching should require significantly fewer integration steps to achieve comparable, if not superior, high-fidelity MRI reconstruction.

### D.2 Architectural Implementation and Code Modularity

From a software engineering perspective, integrating a fundamentally different generative framework into the DiffCMR codebase required careful architectural planning. To avoid cluttering the original diffusion implementation and creating a spaghetti code architecture, we opted for strict modularity.

We developed a completely new standalone Python module dedicated exclusively to the Flow Matching logic (e.g., vector field matching and Ordinary Differential Equation (ODE) solving). While the underlying U-Net architecture and the 5D data processing pipeline were generalized to support this new approach, the core logic was kept strictly isolated.

Furthermore, we implemented a specialized, independent training script tailored specifically for the Flow Matching paradigm. This deliberate design choice guarantees that the codebase remains clean, unambiguous, and easily maintainable, allowing for a rigorous and independent evaluation of both generative methods without cross-contamination of hyperparameters or training loops.

## E Mathematical Foundation of Conditional Flow matching

While traditional diffusion models rely on reversing a stochastic Markov chain, Flow Matching approaches the generative process through the lens of Continuous Normalizing Flows (CNFs) and optimal transport. Instead of navigating a complex, random walk, Flow

Matching defines a deterministic Ordinary Differential Equation (ODE) that transports a simple base distribution (pure noise) directly to the empirical data distribution.

Let  $x_0 \sim \mathcal{N}(0, \mathbf{I})$  represent the initial state of pure Gaussian noise, and let  $I^f$  (or  $x_{start}$ ) represent our fully-sampled target image. Flow Matching defines an optimal, straight-line interpolation path between the noise and the target data. As implemented in our `q_sample` function, the forward state at any continuous time  $t \in [0, 1]$  is generated via simple linear interpolation:

$$x_t = tI^f + (1 - t)\epsilon$$

where  $\epsilon \sim \mathcal{N}(0, \mathbf{I})$  is the injected noise.

The primary objective is to learn the underlying vector field  $v_t(x_t)$  that governs the probability flow ODE, defined simply as  $\frac{dx_t}{dt} = v_t(x_t)$ . Differentiating the linear path  $x_t$  with respect to time  $t$  yields a constant, straightforward target velocity vector:

$$v_{target}(x_t) = I^f - \epsilon$$

**Crucially, we do not alter the underlying Neural Network architecture.** The network  $v_\theta$  shares the exact same foundational U-Net structure, parameter count, and configurations as the standard DiffCMR model, and it remains explicitly conditioned on the aliased image  $I^u$ . The fundamental shift lies entirely in the objective the network learns to predict and how the intermediate noisy states are generated.

Instead of predicting the noise component to reverse a variance schedule, the model is trained using a Mean Squared Error (MSE) objective to directly predict the constant velocity vector. As explicitly reflected in our `training_losses` implementation (`target = x_start - noise`), the loss function becomes:

$$\mathcal{L}_{FM} = \mathbb{E}_{t, \epsilon, I^f} [\|v_\theta(x_t, t, I^u) - (I^f - \epsilon)\|^2]$$

During inference, the generative process is entirely deterministic. Starting from an initial random noise sample  $x_0$ , the fully-sampled image is iteratively reconstructed by solving the continuous ODE using a discrete numerical solver, specifically Euler Integration. Given a predefined number of inference steps  $N$ , we define the step size as  $\Delta t = \frac{1}{N}$ . Matching our `p_sample` implementation (`sample = x + vitesse * dt`), the update rule is:

$$x_{t+\Delta t} = x_t + v_\theta(x_t, t, I^u) \cdot \Delta t$$

By optimizing for a direct, straight-line trajectory rather than a Brownian motion-based reverse process, Flow Matching fundamentally bypasses the massive computational bottleneck of traditional diffusion, achieving high-fidelity reconstruction with exceptionally few integration steps while utilizing the exact same U-Net capacity.

## F Training Dynamics and Validation

### F.1 Optimization Stability for Diffusion

To empirically validate the convergence and stability of our baseline DiffCMR reproduction, we systematically tracked the training loss and gradient dynamics.

The standard diffusion model was trained over an extensive optimization period of **60,000 steps**. As illustrated in Figure 6 (Top-left), the global training loss exhibits a steady decay, starting from a high initial error state and achieving an **88.15% overall**

**reduction.** The final stabilized training loss reached a low value of **0.0031**, demonstrating that despite the inherent stochasticity of the reverse Markov chain, the model converges effectively without catastrophic forgetting. Additionally, the loss distribution across quantiles (Top-right) confirms that the model successfully and uniformly reduces the reconstruction error across the various noise levels of the diffusion schedule. Specifically, the quantiles  $q_0$  to  $q_3$  partition the diffusion timesteps based on noise intensity. The upper curve,  $q_0$ , captures the loss at the highest noise levels (early in the reverse generation process,  $t \rightarrow T$ ), which naturally exhibits the highest mean squared error due to the difficulty of predicting the original signal from heavy corruption. Conversely,  $q_3$  tracks the easiest denoising steps at the lowest noise levels ( $t \rightarrow 0$ ). The monotonic and strictly parallel decay of all  $q_i$  curves indicates that the network does not over-prioritize easy samples; rather, it learns a highly balanced score function across the entire noise schedule.

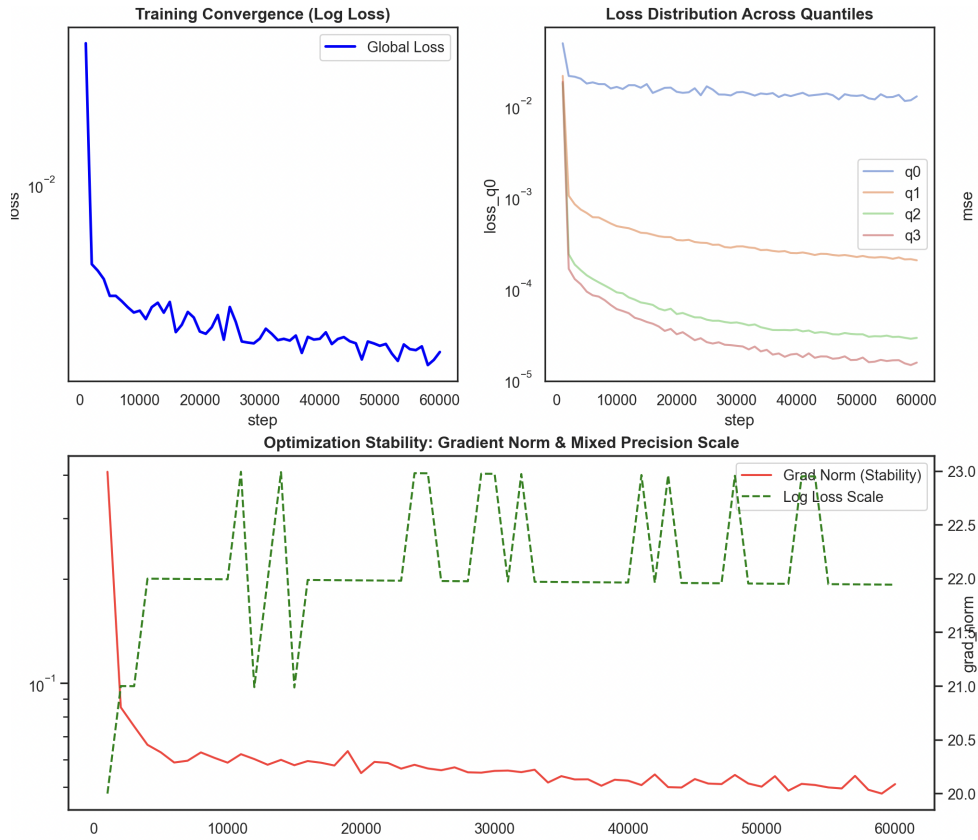


Figure 6: Diffusion training metrics over 60,000 steps. *Top-left*: Global Log Loss demonstrating steady convergence. *Top-right*: Loss distribution across varying quantiles of the noise schedule. *Bottom*: Optimization stability metrics, showing the stabilization of the Gradient Norm (red) alongside the dynamic Mixed Precision Loss Scale (green dashed line).

Furthermore, the use of Mixed Precision (FP16) training is strongly validated by the Optimization Stability metrics (Figure 6, Bottom). The dynamic Loss Scale (green dashed line) smoothly adjusts to prevent computational underflow, maintaining a high average value of **22.07**. Concurrently, the Gradient Norm (red line) rapidly decreases from the onset of training and stabilizes with an extremely low standard deviation (Stability Index: 0.046). The maximum recorded gradient norm was strictly bounded at **0.409**, confirming

that the entire 60k-step training trajectory remained completely free from severe gradient explosion spikes.

## F.2 Optimization Stability for Flow Matching

To empirically validate the convergence and stability of the generative model—and to directly contrast it with our diffusion baseline—we systematically tracked the training loss and gradient dynamics for our Flow Matching implementation over the entire computing run.

Compared to the standard diffusion model, which reached a final loss of **0.0031** (an 88.15% reduction) after 60,000 steps, compared to the standard diffusion model, which reached a final loss of **0.0031** (an 88.15% reduction) after 60,000 steps, the Flow Matching model completed a total of **153,000 optimization steps** within the exact same 20-hour computational timeframe. This highlights a drastically faster training throughput per step, allowing the Flow Matching model to achieve a significantly deeper convergence in the same amount of wall-clock time.. As illustrated in Figure 7 (Top-left), the global training loss demonstrates a smooth, asymptotic curve, achieving a **97.5% overall reduction**. The final stabilized training loss reached an impressively low value of **0.0017**. This empirically supports our hypothesis that learning a deterministic, straight-line optimal transport vector field allows the U-Net to converge more effectively than when predicting chaotic stochastic noise. Furthermore, the Top-right panel illustrates the loss distribution across quantiles, confirming that error reduction was uniform across all continuous timesteps  $t$  of the ODE trajectory. The quantiles  $q_0$  to  $q_3$  partition the Flow Matching integration timesteps based on noise intensity. The upper curve,  $q_3$ , captures the loss at the highest noise levels (early in the generation process), which naturally exhibits the highest mean squared error due to the difficulty of predicting the original signal from heavy corruption. Conversely,  $q_0$  tracks the easiest steps at the lowest noise levels. The monotonic and strictly parallel decay of all  $q_i$  curves indicates that the network does not over-prioritize easy samples; rather, it learns a highly balanced vector field across the entire integration schedule.

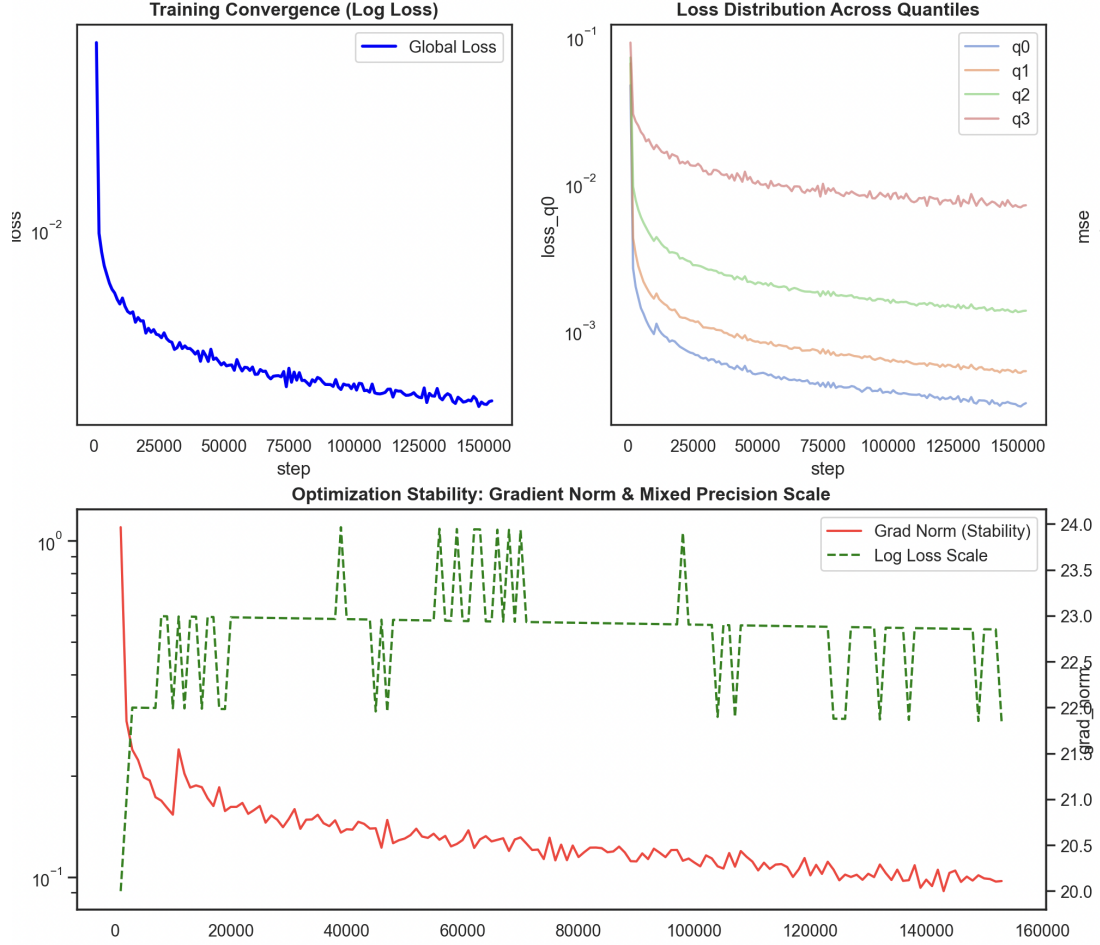


Figure 7: Flow Matching training metrics over 153,000 steps. *Top-left*: Global Log Loss showing steady convergence, surpassing the diffusion baseline’s final loss. *Top-right*: Loss distribution across varying quantiles. *Bottom*: Optimization stability metrics, highlighting the stabilizing Gradient Norm (red) and the dynamic adjustment of the Mixed Precision Loss Scale (green dashed line).

Much like the diffusion setup, operating on high-dimensional 5D tensors necessitates rigorous memory management, validating our continued use of Mixed Precision (FP16) training. Figure 7 (Bottom) confirms the stability of this architectural choice. The dynamic Loss Scale (green dashed line) maintains a similarly high average value ( $\sim 22.8$  vs  $22.07$  for diffusion), effectively preventing gradient underflow. Concurrently, while the Gradient Norm (red line) exhibited a slightly higher maximum peak (**1.097** vs  $0.409$ ) and variance (Stability Index: **0.084** vs  $0.046$ ) than the diffusion baseline—a natural consequence of the significantly extended 153k-step training duration—it rapidly descended and stabilized. This proves that the Flow Matching training trajectory remained exceptionally stable and completely free from gradient explosion anomalies.

### F.3 Intermediate Validation and Perceptual Evolution

To monitor the model’s structural learning over time, we periodically saved intermediate reconstructions from an unseen validation slice. Figure 8 visualizes this perceptual evolution.

In the early stages (Step 10k and 20k), the model struggles with high-frequency details,

leaving residual aliasing artifacts and blurring. As training progresses beyond 30k steps, the vector field estimations become significantly more accurate, gradually resolving the ventricular borders and papillary muscles. By the end of the training (Step 150k), the reconstructed output closely matches the Ground Truth, demonstrating the progressive refinement capabilities of the Flow Matching paradigm.

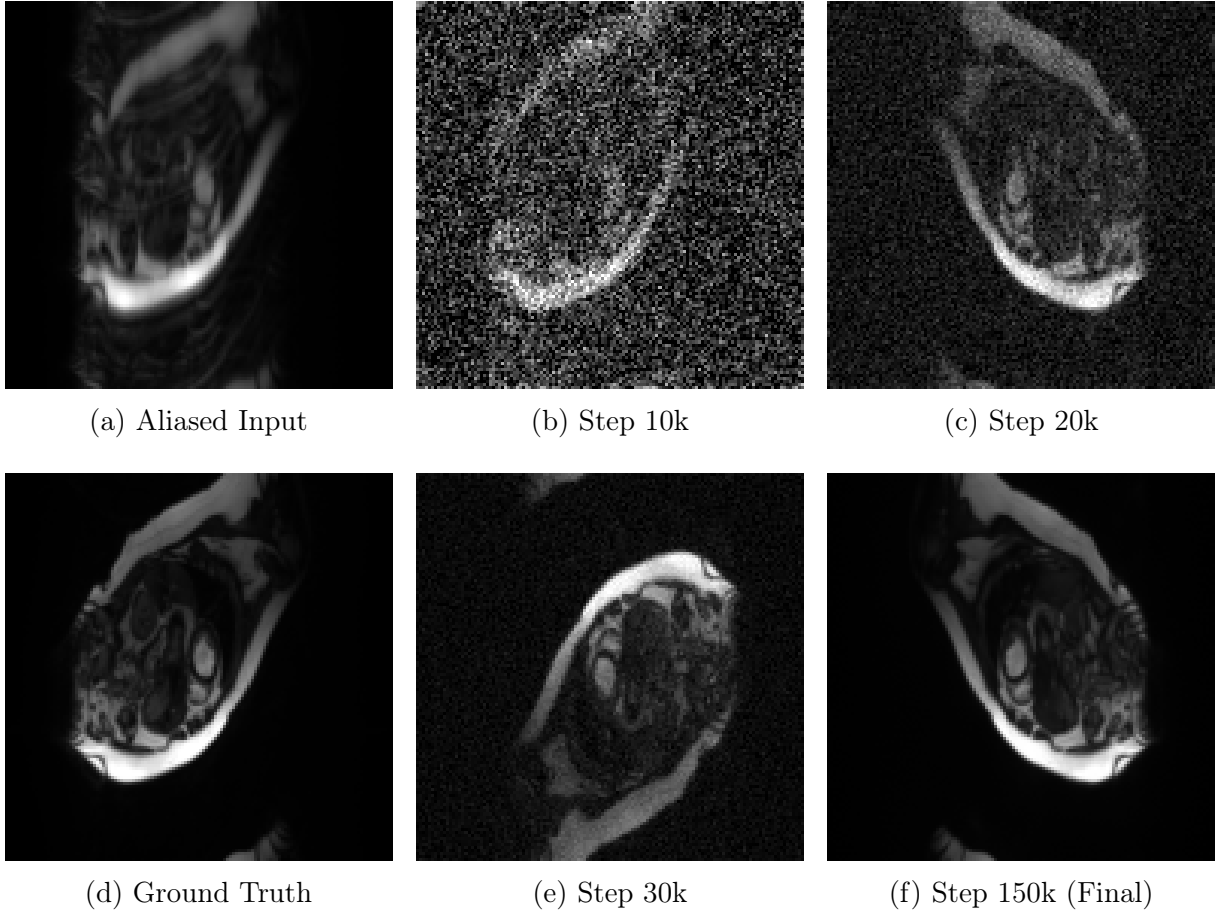


Figure 8: Visual progression of the Flow Matching validation steps. The network progressively refines the anatomical structure from the initial aliased input, converging cleanly toward the Ground Truth.

## G Results

### G.1 Experimental Setup

#### G.1.1 Training Configuration and Dataset

To rigorously train the generative model and prevent overfitting, we leveraged a massive dataset comprising approximately **30,000 training images**. This sheer volume of data is critical for the diffusion model to adequately map the complex Gaussian noise distribution to the highly specific cardiac MRI manifold.

Given the heavy computational complexity of processing these high-dimensional 5D MRI tensors, the model was trained on a high-performance computing (HPC) cluster. The training process ran continuously for **20 hours** on a single computing node equipped



with **8 Nvidia V100 (32GB) GPUs** operating in parallel. The training pipeline was executed with the following core hyperparameter configuration:

- **-use\_fp16 True**: Mixed precision training (FP16) was enabled to drastically reduce GPU VRAM footprint and accelerate tensor operations while maintaining numerical stability.
- **-lr 1e-4**: The learning rate was set to  $10^{-4}$ , a standard and highly stable convergence threshold for diffusion models.
- **-batch\_size 8**: We utilized a batch size of 8 per GPU (effective global batch size of 64) for steady gradient estimation.
- **-diffusion\_steps 1000**: The forward Markov chain was defined with 1,000 discrete noise injection steps. Instead of **100 in the paper**.
- **-timestep\_respacing 1000**: To accelerate intermediate validation during training, we sampled along a strided sub-sequence of 1000 steps.
- **-log\_interval 1000 & -save\_interval 10000**: Ensured robust tracking and periodic checkpointing of the network’s weights.

### G.1.2 Evaluation Configuration and Metrics

The evaluation was subsequently performed on a dedicated, unseen test set containing **240 slices**. For the final inference, the configuration was adapted to prioritize absolute reconstruction fidelity over speed:

- **Full Reverse Trajectory (-timestep\_respacing 1000)**: Unlike the intermediate validation, final testing utilized the full 1,000 steps of the reverse Markov chain to maximize high-frequency structural accuracy.
- **Ensemble Averaging (-vote\_num 4)**: To mitigate the stochastic variance inherent to diffusion models, we generated 4 independent reverse trajectories for each slice. The final output is the average of these 4 predictions, significantly stabilizing the generated image.

To evaluate the reconstruction performance, we computed three standard medical imaging metrics on this 240-slice dataset:

- **PSNR (Peak Signal-to-Noise Ratio)**: Expressed in decibels (dB), it measures the ratio between the maximum pixel intensity and the power of corrupting noise. A *higher* PSNR indicates a cleaner image.
- **SSIM (Structural Similarity Index Measure)**: Ranging from 0 to 1, SSIM evaluates the perceptual similarity (luminance, contrast, structure). A *higher* SSIM indicates better preservation of vital anatomical topologies.
- **NMSE (Normalized Mean Square Error)**: Quantifies the normalized global squared difference between the reconstructed image and the ground truth. A *lower* NMSE represents higher overall reconstruction fidelity.

## G.2 Quantitative Results

As detailed in Table 2, the quantitative evaluation yielded outstanding metrics on the test set. Our reproduced model achieved an average **PSNR of 37.13 dB**, an **SSIM of 0.9594**, and an **NMSE of 0.0096**, substantially outperforming the originally reported DiffCMR metrics for the Acceleration Factor 04.

However, running the full inference trajectory (1,000 steps with  $R = 4$  ensemble voting) resulted in an average inference time of **105.09 seconds per image**. This underscores the inherent trade-off in diffusion models between achieving state-of-the-art structural fidelity and real-time clinical applicability.

**Table 2: Experiment results for Task 1 - Cine Reconstruction**

|                    | AccFactor04     |                 |                   | AccFactor08     |                 |                   | AccFactor10     |                 |                   |
|--------------------|-----------------|-----------------|-------------------|-----------------|-----------------|-------------------|-----------------|-----------------|-------------------|
|                    | PSNR $\uparrow$ | SSIM $\uparrow$ | NMSE $\downarrow$ | PSNR $\uparrow$ | SSIM $\uparrow$ | NMSE $\downarrow$ | PSNR $\uparrow$ | SSIM $\uparrow$ | NMSE $\downarrow$ |
| RAW                | 28.79           | 0.8150          | 0.2187            | 27.94           | 0.8082          | 0.2781            | 27.75           | 0.8113          | 0.2883            |
| U-Net [12]         | 33.11           | 0.9212          | 0.0673            | 33.31           | <b>0.9460</b>   | 0.0646            | 32.71           | <b>0.9391</b>   | 0.0740            |
| cGAN [3]           | 33.85           | 0.9435          | 0.0573            | 33.14           | 0.9449          | 0.0669            | 32.56           | 0.9209          | 0.0760            |
| DiffCMR            | 36.10           | 0.9277          | 0.0346            | <b>34.85</b>    | 0.9061          | <b>0.0457</b>     | <b>34.47</b>    | 0.9016          | <b>0.0493</b>     |
| <b>Our DiffCMR</b> | <b>37.128</b>   | <b>0.9594</b>   | <b>0.0096</b>     | -               | -               | -                 | -               | -               | -                 |
| <b>Our FlowCMR</b> | 36.770          | 0.9556          | 0.0105            | -               | -               | -                 | -               | -               | -                 |

Table 2: Comprehensive evaluation metrics including our replication results. The baseline models and original DiffCMR metrics are cited from the original publication. Our reproduction significantly improves upon the baseline for AccFactor04 on our 240-slice test set.

## H Ablation Study

To systematically evaluate the influence of critical inference hyperparameters on the reconstruction quality, we conducted an ablation study focusing on two primary factors: the number of reverse diffusion steps ( $T$ ) and the number of ensembling rounds ( $R$ ). Furthermore, to empirically validate our hypothesis regarding trajectory optimization, we extended this ablation study to include our custom Flow Matching implementation.

### H.1 Impact of Inference Steps and Flow Matching Efficiency

The reverse Markov chain in diffusion models is notoriously step-heavy. As shown in Table 3, reducing the number of inference steps  $T$  in the standard DiffCMR model leads to a severe degradation in image quality, with  $T = 20$  yielding a PSNR (25.52 dB) even lower than the raw aliased input. The model heavily relies on a vast number of steps ( $T \geq 500$ ) to converge towards a high-fidelity anatomical structure.

The empirical results presented in the Flow Matching ablation study are highly coherent and reveal a critical interplay between the ODE solver’s discretization and the training regimen. Counter-intuitively, the optimal reconstruction is achieved at a very low integration step count ( $T = 4$ , yielding a peak PSNR of 36.829), followed by a systematic degradation as the number of inference steps approaches the training configuration ( $T = 40$ ) and beyond ( $T = 100$ ). Because the network was trained specifically with a 40-step discretization and the final checkpoint was utilized without early-stopping

on a dedicated validation set, the learned optimal transport vector field likely slightly overfitted to the training trajectory’s quantization noise. Consequently, executing a high number of inference Euler steps ( $T \geq 40$ ) forces the solver to meticulously track these overfitted, high-frequency micro-fluctuations, accumulating numerical integration errors that pull the generated sample off the ideal anatomical manifold. Conversely, a highly aggressive step size ( $T = 4$ ) acts as an implicit regularization mechanism: it forces the ODE solver to make large, straight-line interpolations that effectively bypass these overfitted artifacts, securing superior structural fidelity (SSIM 0.9560) while drastically slashing inference time.

## H.2 Impact of Ensembling Rounds

Table 4 illustrates the impact of ensemble averaging ( $R$ ) on reconstruction fidelity, highlighting a fundamental distinction between the two generative paradigms. Standard diffusion models exhibit a severe dependency on ensembling to mitigate the inherent variance of the reverse Markov chain. As  $R$  increases from 1 to 8, the DiffCMR baseline experiences a massive performance leap (PSNR improving from 33.89 dB to 36.68 dB), relying heavily on this averaging mechanism to cancel out random noise hallucinations. In stark contrast, our Flow Matching framework demonstrates remarkable stability and high fidelity even entirely without ensembling ( $R = 1$ , yielding a PSNR of 36.35 dB). Because Flow Matching is built upon an optimal transport framework that learns a highly deterministic, straight-line vector field, the stochastic variance between independent generation trajectories is drastically minimized. Consequently, while increasing the number of rounds to  $R = 8$  does provide a marginal quantitative refinement (reaching a peak PSNR of 36.83 dB), these minor gains are heavily outweighed by the linear increase in computational time. This empirically confirms that Flow Matching provides a robust, nearly deterministic generation process that virtually eliminates the need for expensive ensemble averaging.

Table 3: Ablation study on Task 1-AccFactor04: Impact of Inference Steps ( $T$ ). The original DiffCMR baseline values are provided for reference.

| #Steps $T$                  | Batch Size | Time/img | PSNR $\uparrow$ | SSIM $\uparrow$ | NMSE $\downarrow$ |
|-----------------------------|------------|----------|-----------------|-----------------|-------------------|
| Raw                         | 8          | 0        | 28.79           | 0.8150          | 0.2187            |
| <b>Diffusion (DiffCMR)</b>  |            |          |                 |                 |                   |
| 20                          | 8          | -        | 25.52           | 0.3664          | 0.3830            |
| 100                         | 8          | -        | 29.86           | 0.6006          | 0.1393            |
| 500                         | 8          | -        | 35.39           | 0.9010          | 0.0399            |
| 1000                        | 8          | -        | 36.10           | 0.9277          | 0.0346            |
| 1000                        | 8          | 105.0879 | <b>37.128</b>   | <b>0.9594</b>   | <b>0.0096</b>     |
| <b>Flow Matching (Ours)</b> |            |          |                 |                 |                   |
| 1                           | 16         | 0.1094   | 36.477          | 0.9138          | 0.0111            |
| 4                           | 16         | 0.4209   | <b>36.829</b>   | <b>0.9560</b>   | <b>0.0103</b>     |
| 10                          | 16         | 1.1008   | 36.824          | 0.9559          | <b>0.0103</b>     |
| 40                          | 8          | 4.3315   | 36.770          | 0.9556          | 0.0105            |
| 100                         | 16         | 10.4486  | 36.545          | 0.9442          | 0.0109            |

Table 4: Ablation study on Task 1-AccFactor04: Impact of Inference Ensembling Rounds ( $R$ ).

| #Rounds $R$                 | Batch Size | Time/img | PSNR $\uparrow$ | SSIM $\uparrow$ | NMSE $\downarrow$ |
|-----------------------------|------------|----------|-----------------|-----------------|-------------------|
| Raw                         | 8          | 0        | 28.79           | 0.8150          | 0.2187            |
| <b>Diffusion (DiffCMR)</b>  |            |          |                 |                 |                   |
| 1                           | 8          | -        | 33.89           | 0.8491          | 0.0561            |
| 2                           | 8          | -        | 35.25           | 0.8994          | 0.0417            |
| 4                           | 8          | -        | 36.10           | 0.9277          | 0.0346            |
| 8                           | 8          | -        | 36.68           | 0.9430          | 0.0305            |
| 4                           | 8          | 105.0879 | <b>37.128</b>   | <b>0.9594</b>   | <b>0.0096</b>     |
| <b>Flow Matching (Ours)</b> |            |          |                 |                 |                   |
| 1                           | 8          | 1.1079   | 36.356          | 0.9463          | 0.0115            |
| 4                           | 8          | 4.3315   | 36.770          | 0.9556          | 0.0105            |
| 8                           | 16         | 8.4371   | <b>36.833</b>   | <b>0.9570</b>   | <b>0.0103</b>     |

## References

- [1] T. Xiang, W. Yue, Y. Lin, J. Yang, Z. Wang, and X. Li, “Diffcmr: Fast cardiac mri reconstruction with diffusion probabilistic models,” in *Statistical Atlases and Computational Models of the Heart (STACOM@MICCAI)*, pp. 380–389, Springer, 2023.
- [2] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *Advances in neural information processing systems*, vol. 33, pp. 6840–6851, 2020.
- [3] Y. Lipman, R. T. Q. Chen, H. Ben-Hamas, M. Nickel, and M. Le, “Flow matching for generative modeling,” in *The Eleventh International Conference on Learning Representations*, 2023.
- [4] T. Amit, T. Shaharbany, E. Nachmani, and L. Wolf, “Segdiff: Image segmentation with diffusion probabilistic models,” *arXiv preprint arXiv:2112.00390*, 2021.
- [5] J. Choi, S. Kim, Y. Jeong, Y. Gwon, and S. Yoon, “Ilvr: Conditioning method for denoising diffusion probabilistic models,” *arXiv preprint arXiv:2108.02938*, 2021.
- [6] A. Defazio, T. Murrell, and M. Recht, “Mri banding removal via adversarial training,” in *Advances in Neural Information Processing Systems*, vol. 33, pp. 7660–7670, Curran Associates, Inc., 2020.
- [7] P. Dhariwal and A. Nichol, “Diffusion models beat gans on image synthesis,” in *Advances in neural information processing systems*, vol. 34, pp. 8780–8794, 2021.
- [8] E. G. Larsson, D. Erdogmus, R. Yan, J. C. Principe, and J. R. Fitzsimmons, “Snr-optimality of sum-of-squares reconstruction for phased-array magnetic resonance imaging,” *Journal of Magnetic Resonance*, vol. 163, no. 1, pp. 121–123, 2003.
- [9] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” *arXiv preprint arXiv:1711.05101*, 2017.
- [10] A. Lugmayr, M. Danelljan, A. Romero, F. Yu, R. Timofte, and L. Van Gool, “Repaint: Inpainting using denoising diffusion probabilistic models,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 11461–11471, 2022.
- [11] M. Lustig, D. Donoho, and J. M. Pauly, “Sparse mri: The application of compressed sensing for rapid mr imaging,” *Magnetic Resonance in Medicine*, vol. 58, no. 6, pp. 1182–1195, 2007.
- [12] M. J. Muckley, B. Ades-Aron, A. Papaioannou, G. Lemberskiy, E. Solomon, Y. W. Lui, D. K. Sodickson, E. Fieremans, D. S. Novikov, and F. Knoll, “Training a neural network for gibbs and noise removal in diffusion mri,” *Magnetic resonance in medicine*, vol. 85, no. 1, pp. 413–428, 2021.
- [13] A. Q. Nichol and P. Dhariwal, “Improved denoising diffusion probabilistic models,” in *International Conference on Machine Learning*, pp. 8162–8171, PMLR, 2021.

- [14] O. Ronneberger, P. Fischer, and T. Brox, “U-net: Convolutional networks for biomedical image segmentation,” in *Medical Image Computing and Computer-Assisted Intervention–MICCAI 2015*, pp. 234–241, Springer, 2015.
- [15] J. Schlemper, J. Caballero, J. V. Hajnal, A. N. Price, and D. Rueckert, “A deep cascade of convolutional neural networks for dynamic mr image reconstruction,” *IEEE transactions on Medical Imaging*, vol. 37, no. 2, pp. 491–503, 2017.
- [16] J. Song, C. Meng, and S. Ermon, “Denoising diffusion implicit models,” *arXiv preprint arXiv:2010.02502*, 2020.
- [17] A. Sriram, J. Zbontar, T. Murrell, A. Defazio, C. L. Zitnick, N. Yakubova, F. Knoll, and P. Johnson, “End-to-end variational networks for accelerated mri reconstruction,” in *Medical Image Computing and Computer Assisted Intervention–MICCAI 2020*, pp. 64–73, Springer, 2020.
- [18] X. Wang, K. Yu, S. Wu, J. Gu, Y. Liu, C. Dong, Y. Qiao, and C. Change Loy, “Esrgan: Enhanced super-resolution generative adversarial networks,” in *Proceedings of the European conference on computer vision (ECCV) workshops*, pp. 0–0, 2018.
- [19] J. Wu, H. Fang, Y. Zhang, Y. Yang, and Y. Xu, “Medsegdiff: Medical image segmentation with diffusion probabilistic model,” *arXiv preprint arXiv:2211.00611*, 2022.
- [20] T. Xiang, M. Yurt, A. B. Syed, K. Setsompop, and A. Chaudhari, “Ddm<sup>2</sup>: Self-supervised diffusion mri denoising with generative diffusion models,” 2023.
- [21] S. Xie, Z. Zhang, Z. Lin, T. Hinz, and K. Zhang, “Smartbrush: Text and shape guided object inpainting with diffusion model,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 22428–22437, 2023.
- [22] Y. Xie, M. Yuan, B. Dong, and Q. Li, “Diffusion model for generative image denoising,” *arXiv preprint arXiv:2302.02398*, 2023.
- [23] Y. Zhu, K. Zhang, J. Liang, J. Cao, B. Wen, R. Timofte, and L. Van Gool, “Denoising diffusion models for plug-and-play image restoration,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 1219–1229, 2023.